TELECOMMUNICATION ENGINEERING LABORATORY

A Study Guide to Four Python Simulation Experiments

Experiment 1	Pulse Code Modulation (PCM)
Experiment 2	Direct-Sequence Spread Spectrum (DSSS)
Experiment 3	Frequency-Hopping Spread Spectrum (FHSS)
Experiment 4	Convolutional Coding and Viterbi Decoding

This guide explains the theory, signal flow, key equations, Python modules, plot interpretation, and common viva questions for each experiment.

The simulations are educational baseband models. Adjust parameters to match your laboratory manual.

How to Use This Guide

Each chapter follows this structure: objective, theory, block-level flow, equations, implementation overview, interpretation, and viva questions. The example scripts use NumPy for numerical calculations and Matplotlib for plots.

Requirements

• Python 3.9 or later.

• Install dependencies: `python -m pip install numpy matplotlib`.

• Run one experiment, for example: `python pcm_simulation.py`.

• The random seeds make the examples reproducible. Changing a seed changes the random data/noise.

Common terminology

SNR: signal-to-noise ratio, often expressed as 10 log10(Psignal/Pnoise) dB. **BER:** number of incorrectly detected bits divided by the number of compared bits.

Model limitations: these simulations simplify real communication systems. They do not model every RF, synchronization, hardware, or standards-specific detail.

Experiment 1 — Pulse Code Modulation (PCM)

Objective

Sample an analog-like message, quantize each sample to one of a finite number of levels, encode each quantization index in binary, and reconstruct an approximation of the original signal.

Theory

PCM converts an analog message into a digital representation. The sampler measures the signal at regular intervals. The quantizer maps each sample to a permitted amplitude level. The encoder turns the level index into a fixed-width binary word. A receiver decodes the words and reconstructs a waveform from the quantized samples.

Block-level flow

```
Analog message → Sampler → Quantizer → Binary encoder → Bitstream → Decoder → Reconstruction
```

Key equations

For b bits per sample and a uniform quantizer covering –A to +A:

```
Number of levels: L = 2^b
Quantization step: Δ = 2A/L
PCM bit rate: Rb = fs × b
```

Here fs is the sampling rate. More bits give more quantization levels and generally smaller quantization error, but also increase the bit rate.

Python function roles

• `generate_message()`: generates a test waveform from sinusoidal components.

• `sample_signal()`: obtains regularly spaced samples.

• `quantize_signal()`: maps amplitudes to quantization indices and levels.

• `encode_pcm()`: converts each index into a binary word.

• `decode_pcm()`: groups bits into words and recovers indices.

• `reconstruct_signal()`: interpolates quantized samples for display.

Plot interpretation

The first plot shows the original message and sample points. The second compares original, quantized, and reconstructed signals. The third displays part of the PCM bitstream. Quantized samples should fall on discrete amplitude levels.

Example settings

Parameter	Example	Meaning
Bits/sample	3	8 quantization levels
Sampling rate	30 Hz	30 samples/second
Bit rate	90 bit/s	30 × 3

Viva questions

• Why sample? To represent a continuous-time signal by discrete-time values.

• What is quantization? Mapping sample amplitudes to a finite set of levels.

• What happens when bit depth increases? More levels and usually less quantization error, but a higher bit rate.

• What is quantization error? The difference between a sample and its quantized value.

Experiment 2 — Direct-Sequence Spread Spectrum (DSSS)

Objective

Spread BPSK information symbols with a pseudo-noise (PN) chip sequence, pass the spread signal through a noisy/interfering channel, and recover the data by correlation.

Theory

DSSS multiplies each information symbol by a faster sequence of chips. A single bit is represented by several chips, spreading the signal over a wider bandwidth. A receiver that knows the same PN sequence can correlate each received chip block with that sequence to despread and detect the data.

Block-level flow

```
Bits → BPSK mapping → PN spreading → Channel (AWGN + narrowband interference) →
Correlation/despreading → Bit decisions
```

Key equations

BPSK mapping: a[i] = 2b[i] – 1, so bit 0 maps to –1 and bit 1 to +1.

```
Spread chips: x[iN+k] = a[i]c[k]
Correlation: z[i] = (1/N) Σ r[iN+k]c[k]
Decision: bit 1 if z[i] ≥ 0; otherwise bit 0
Approximate processing gain: Gp(dB) = 10 log10(N)
```

N is the number of chips per bit; c[k] is the PN sequence. The example adds Gaussian noise and a sinusoidal narrowband interferer.

Python function roles

• `generate_bits()`: creates random data.

• `bpsk_modulate()`: maps bits to –1/+1 symbols.

• `generate_pn_sequence()`: creates the spreading chips.

• `spread_signal()`: multiplies symbols by PN chips.

• `add_awgn()` and `add_interference()`: model channel impairments.

• `despread_signal()`: correlates received chips and makes bit decisions.

Plot interpretation

The spread chip sequence changes faster than the original data. The received sequence is disturbed by noise and interference. The final plot compares the original and detected bits; mismatches determine the bit error rate (BER).

Limitations

The receiver is assumed to know the exact PN sequence and chip timing. This is a simplified baseband model, not a full RF implementation. Processing gain is a theoretical indicator; real rejection depends on synchronization and channel conditions.

Viva questions

• What is a chip? An element of the faster spreading sequence.

• Why use a PN sequence? To spread the data and enable synchronized despreading.

• What is processing gain? A measure related to spread bandwidth relative to information bandwidth.

• What does the receiver need? The correct PN sequence and suitable timing synchronization.

Experiment 3 — Frequency-Hopping Spread Spectrum (FHSS)

Objective

Transmit BPSK data using a carrier that changes frequency according to a hopping pattern, then recover bits using the known pattern.

Theory

FHSS changes the carrier among a set of frequencies according to a predetermined pattern. In this experiment, one carrier is used for each bit interval. The receiver knows the hop frequency for each interval and correlates the received waveform with the corresponding local carrier.

Block-level flow

```
Bits + hop pattern → BPSK on selected carrier → AWGN channel → Synchronized carrier correlation → Detected bits
```

Key equations

During interval i, a[i] is the BPSK symbol and f[i] is the hop frequency:

```
s[i](t) = a[i] cos(2πf[i]t)
q[i] = Σ r[i,n] cos(2πf[i]n/fs)
Decision: bit 1 if q[i] ≥ 0; otherwise bit 0
```

Python function roles

• `generate_bits()`: creates information bits.

• `generate_hop_pattern()`: selects a frequency for each bit.

• `generate_fhss_signal()`: generates the frequency-hopped BPSK waveform.

• `add_awgn()`: adds Gaussian noise.

• `detect_fhss()`: correlates each interval with the expected carrier.

Plot interpretation

The hopping plot shows which carrier frequency is selected for each bit. The waveform plot shows the carrier changing between intervals. The last plot compares transmitted and detected data.

Limitations

The model assumes perfect timing and exact knowledge of the hop pattern. It does not model frequency synthesizer settling, hop acquisition, synchronization errors, or realistic RF hardware.

Viva questions

• What is frequency hopping? Changing the carrier among a set of frequencies according to a pattern.

• Why must the receiver synchronize? It must use the correct frequency in each interval.

• How does FHSS differ from DSSS? FHSS changes carrier frequency; DSSS multiplies data by a chip sequence.

• What determines the hop? The hopping sequence and timing.

Experiment 4 — Convolutional Coding and Viterbi Decoding

Objective

Add redundancy to data using a convolutional encoder, introduce random bit errors, and recover the most likely information sequence using Viterbi decoding.

Theory

A convolutional encoder generates output bits using the current input and a memory of previous inputs. The rate-1/2 encoder produces two coded bits per input bit. The Viterbi decoder searches paths through the encoder's state trellis and selects a path with the smallest accumulated Hamming distance from the received sequence.

Block-level flow

```
Information bits → Convolutional encoder → Binary symmetric channel → Viterbi decoder → Recovered bits
```

Configuration and key concepts

The example uses constraint length K = 3 and generator polynomials 111 and 101 (binary). It has 2^(K–1) = 4 states and rate R = 1/2. Two zero tail bits are appended to return the encoder to state zero.

```
Branch metric = number of differing bits between a candidate encoder output and the received output.
Path metric = accumulated branch metric along a trellis path.
Viterbi decoding selects the minimum-metric path.
```

Python function roles

• `convolutional_encode()`: generates coded bits and optional tail bits.

• `binary_symmetric_channel()`: independently flips bits with a specified probability.

• `viterbi_decode()`: calculates path metrics, stores predecessor decisions, and traces back the selected path.

• `calculate_ber()`: compares original and decoded information bits.

Plot interpretation

The encoded stream is about twice as long as the input stream, plus termination bits. The channel may flip some coded bits. Viterbi decoding uses code redundancy to correct some errors, although successful correction is not guaranteed for every noise realization.

Evaluation note

A single short run is not enough to characterize error-correction performance. For a fair study, evaluate many trials over a range of channel error probabilities or SNR values. The script reports BER for one reproducible run.

Viva questions

• Why use channel coding? To add redundancy that helps detect or correct errors.

• What does rate 1/2 mean? Two coded bits are generated per information bit.

• What is constraint length? It describes the encoder's memory span.

• What is the Viterbi algorithm? A dynamic-programming method for finding a minimum-metric trellis path.

• Why append tail bits? To drive the encoder to a known final state.

Comparison and Practical Checklist

Experiment	Main idea	Receiver	Measure
PCM	Digital representation of sampled amplitudes	Decode words and reconstruct	Quantization error / bit rate
DSSS	Spread data with PN chips	PN correlation	BER / processing gain
FHSS	Change carrier by a hop pattern	Carrier correlation per interval	BER
Convolutional code	Add structured redundancy	Viterbi trellis decoding	Decoded BER

Common troubleshooting

• If an import fails, check that the experiment files are saved in the same directory.

• If a package is missing, run `python -m pip install numpy matplotlib`.

• If BER is unexpected, check SNR/error probability, random seed, and transmitter/receiver sequence alignment.

• If plots look crowded, reduce the number of displayed samples or bits.

• If the instructor's lab manual uses different settings, change the code parameters to match it.

Suggested lab-report structure

• Objective and brief theory.

• Block diagram and key equations.

• Algorithm or modular function descriptions.

• Parameter table and output plots.

• Observations, performance measures, and conclusion.

End of guide